

Funciones autorizadas - Hilos POSIX (resumen con ejemplos y salida en terminal)

Generado automáticamente: resumen en español de las funciones que indicaste, con ejemplos de uso y la salida esperada en terminal. Guarda este documento como referencia para tu proyecto concurrente.

Tabla de contenidos

1. Creación y gestión de hilos (pthread_create, pthread_join)
 2. Mutexes (pthread_mutex_*)
 3. Variables de condición (pthread_cond_*)
 4. Tiempo y E/S (gettimeofday, usleep, write)
 5. Memoria dinámica (malloc, free)
 6. Impresión y utilidades de cadena (printf, fprintf, strcmp, strlen, atoi, memset)
-

1) pthread_create

Firma:

```
int pthread_create(pthread_t *thread,
                  const pthread_attr_t *attr,
                  void *(*start_routine)(void *),
                  void *arg);
```

Crea un nuevo hilo que ejecuta `start_routine` con el argumento `arg`. Devuelve 0 en éxito o un código de error.

Ejemplo

```
#include <pthread.h>
#include <stdio.h>

void* funcion_hilo(void* arg) {
    int id = *(int*)arg;
    printf("Hola desde el hilo %d\n", id);
    return NULL;
}

int main() {
    pthread_t hilo;
    int id = 1;

    pthread_create(&hilo, NULL, funcion_hilo, &id);
    pthread_join(hilo, NULL);
}
```

```
    return 0;
}
```

Salida en terminal:

```
Hola desde el hilo 1
```

2) pthread_join

Firma:

```
int pthread_join(pthread_t thread, void **retval);
```

Espera a que el hilo `thread` termine y opcionalmente recoge el valor de retorno.

Ejemplo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void* tarea_hilo(void* arg) {
    printf("Realizando tarea...\n");
    sleep(1);
    int* resultado = malloc(sizeof(int));
    *resultado = 42;
    return resultado;
}

int main() {
    pthread_t hilo;
    void* retorno;

    pthread_create(&hilo, NULL, tarea_hilo, NULL);
    pthread_join(hilo, &retorno);

    printf("Resultado del hilo: %d\n", *(int*)retorno);
    free(retorno);
    return 0;
}
```

Salida en terminal:

```
Realizando tarea...
Resultado del hilo: 42
```

3) pthread_mutex_init / pthread_mutex_destroy

Firmas:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Inicializa y destruye un mutex. También existe el inicializador estático PTHREAD_MUTEX_INITIALIZER.

pthread_mutex_lock / pthread_mutex_unlock

Firmas:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Bloquea y desbloquea el mutex para proteger secciones críticas.

Ejemplo (contador compartido)

```
#include <pthread.h>
#include <stdio.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int contador = 0;

void* incrementar(void* arg) {
    for(int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&mutex);
        contador++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t hilo1, hilo2;

    pthread_create(&hilo1, NULL, incrementar, NULL);
    pthread_create(&hilo2, NULL, incrementar, NULL);

    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    printf("Contador final: %d\n", contador);
    pthread_mutex_destroy(&mutex);
}
```

```
    return 0;
}
```

Salida en terminal (ejemplo esperado):

```
Contador final: 200000
```

(Sin el mutex la salida sería indeterminada y probablemente menor a 200000.)

4) pthread_cond_init / pthread_cond_wait / pthread_cond_timedwait / pthread_cond_broadcast / pthread_cond_destroy

Firmas clave:

```
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex,
                             const struct timespec *abstime);
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_destroy(pthread_cond_t *cond);
```

Las variables de condición permiten que un hilo espere hasta que otra parte notifique un evento. `pthread_cond_wait` desbloquea el mutex y suspende el hilo; al despertarse, vuelve a bloquear el mutex.

Ejemplo: Productor - Consumidor simple

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int buffer = 0;
int datos_disponibles = 0;

void* productor(void* arg) {
    for(int i = 0; i < 5; i++) {
        pthread_mutex_lock(&mutex);
        buffer = i+1;
        datos_disponibles = 1;
        printf("Producido: %d\n", i+1);
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
        usleep(100000);
    }
    return NULL;
}
```

```

void* consumidor(void* arg) {
    for(int i = 0; i < 5; i++) {
        pthread_mutex_lock(&mutex);
        while(!datos_disponibles) {
            pthread_cond_wait(&cond, &mutex);
        }
        printf("Consumido: %d\n", buffer);
        datos_disponibles = 0;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    pthread_create(&prod, NULL, productor, NULL);
    pthread_create(&cons, NULL, consumidor, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_cond_destroy(&cond);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Salida en terminal (posible orden):

```

Producido: 1
Consumido: 1
Producido: 2
Consumido: 2
Producido: 3
Consumido: 3
Producido: 4
Consumido: 4
Producido: 5
Consumido: 5

```

5) gettimeofday

Firma:

```

int gettimeofday(struct timeval *tv, struct timezone *tz);

```

Obtiene el tiempo actual con precisión en microsegundos. `tv->tv_sec` son segundos desde epoch y `tv->tv_usec` microsegundos.

Ejemplo

```
#include <sys/time.h>
#include <stdio.h>

int main() {
    struct timeval tiempo;
    gettimeofday(&tiempo, NULL);
    printf("Segundos: %ld\nMicrosegundos: %ld\n", tiempo.tv_sec, tiempo.tv_usec);
    return 0;
}
```

Salida en terminal (ejemplo):

```
Segundos: 1622551234
Microsegundos: 123456
```

6) usleep

Firma:

```
int usleep(useconds_t usec);
```

Pausa la ejecución el número de microsegundos indicado.

Ejemplo

```
#include <unistd.h>
#include <stdio.h>

int main() {
    printf("Inicio\n");
    usleep(2000000); // 2 segundos
    printf("Pasaron 2 segundos\n");
    return 0;
}
```

Salida en terminal:

```
Inicio
Pasaron 2 segundos
```

7) write

Firma:

```
ssize_t write(int fd, const void *buf, size_t count);
```

Escribe `count` bytes desde `buf` al descriptor de archivo `fd` (ej. `STDOUT_FILENO`).

Ejemplo

```
#include <unistd.h>
#include <string.h>

int main() {
    char mensaje[] = "Hola Mundo\n";
    write(STDOUT_FILENO, mensaje, strlen(mensaje));
    return 0;
}
```

Salida en terminal:

```
Hola Mundo
```

8) malloc / free

Firmas:

```
void *malloc(size_t size);
void free(void *ptr);
```

Reserva y libera memoria dinámica en el heap.

Ejemplo

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    int* arreglo = (int*)malloc(5 * sizeof(int));
    if(arreglo != NULL) {
        for(int i = 0; i < 5; i++) {
            arreglo[i] = i * 2;
            printf("%d ", arreglo[i]);
        }
        printf("\n");
        free(arreglo);
    }
}
```

```
    return 0;
}
```

Salida en terminal:

```
0 2 4 6 8
```

9) printf / fprintf

Firmas:

```
int printf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
```

Imprime texto formateado a la salida estándar o a un stream (ej. stderr).

Ejemplo

```
#include <stdio.h>

int main() {
    printf("Mensaje a pantalla\n");
    fprintf(stderr, "Mensaje a errores\n");
    return 0;
}
```

Salida en terminal (stdout/ stderr):

```
Mensaje a pantalla
Mensaje a errores
```

10) strcmp

Firma:

```
int strcmp(const char *s1, const char *s2);
```

Compara dos cadenas byte a byte; devuelve 0 si son iguales, <0 si s1 < s2, >0 si s1 > s2.

Ejemplo

```
#include <string.h>
#include <stdio.h>

int main() {
    char str1[] = "hola";
    char str2[] = "hola";
```

```

char str3[] = "adios";

if(strcmp(str1, str2) == 0) printf("str1 y str2 son iguales\n");
if(strcmp(str1, str3) < 0) printf("str1 va antes que str3\n");
return 0;
}

```

Salida en terminal:

```

str1 y str2 son iguales
str1 va antes que str3

```

11) strlen

Firma:

```

size_t strlen(const char *s);

```

Devuelve la longitud de la cadena (sin contar el terminador '\0').

Ejemplo

```

#include <string.h>
#include <stdio.h>

int main() {
    char texto[] = "Programación";
    printf("Longitud de '%s': %zu\n", texto, strlen(texto));
    return 0;
}

```

Salida en terminal:

```

Longitud de 'Programación': 12

```

12) atoi

Firma:

```

int atoi(const char *nptr);

```

Convierte una cadena a entero. Si la cadena no es un número válido, el comportamiento puede ser 0 o indefinido; para conversiones robustas usar `strtol`.

Ejemplo

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    char numero_str[] = "12345";
    int numero = atoi(numero_str);
    printf("Número convertido: %d\n", numero + 10);
    return 0;
}
```

Salida en terminal:

```
Número convertido: 12355
```

13) memset

Firma:

```
void *memset(void *s, int c, size_t n);
```

Rellena el bloque de memoria apuntado por *s* con el byte *c* durante *n* bytes.

Ejemplo

```
#include <string.h>
#include <stdio.h>

int main() {
    char buffer[20];
    memset(buffer, 0, sizeof(buffer));
    strcpy(buffer, "Texto");
    printf("Buffer: %s\n", buffer);
    return 0;
}
```

Salida en terminal:

```
Buffer: Texto
```

Notas finales: siempre comprueba códigos de retorno en producción (p. ej. el valor devuelto por `pthread_create`, `pthread_mutex_lock`, etc.). Libera recursos (mutexes, condvars, memoria) para evitar fugas. Para conversiones numéricas más seguras usa `strtol`/`strtod`. Evita deadlocks con orden consistente de adquisición de mutexes y usa `pthread_cond_timedwait` si necesitas tiempos de espera.

Si quieres, puedo generar también una versión reducida para impresión o añadir diagramas del flujo entre hilos.